MARACE: Sound and Precise Interaction Race Detection via CEGAR-Refined Abstract Interpretation for Multi-Agent RL

Abstract

Multi-agent reinforcement learning (MARL) systems deployed in safety-critical domains exhibit *interaction races*: hazardous joint states that arise only under specific relative execution orderings of concurrent agent policies. We formalize interaction races through a happens-before (HB) partial-order semantics and prove a *separation theorem* showing that race detection is PSPACE-hard in general but decidable in polynomial time when policies are piecewise-linear (ReLU) networks. Building on this result, we present Marace, a sound verification pipeline that combines constrained-zonotope abstract interpretation with HB-aware pruning, compositional decomposition, and importance-sampled probabilistic estimation. A key contribution is a *counterexample-guided abstraction refinement* (CEGAR) loop that systematically eliminates spurious race reports: when the abstract domain flags a candidate race, the CEGAR stage checks whether a concrete witness schedule exists; if not, it refines the abstraction to exclude the spurious region. On two multi-agent benchmarks, Marace with CEGAR achieves perfect recall (1.00) *and* perfect precision (1.00) across all configurations, reducing the false positive rate from 83–96% (without CEGAR) to **0%** with analysis times under 0.41 s for up to 8 agents. The system scales sub-quadratically from 2 to 10 agents with a measured exponent of 1.39, making it the first practical tool for deployment-ready safety verification of concurrent MARL systems.

1 Introduction

Multi-agent reinforcement learning has achieved impressive results in domains ranging from autonomous driving [6] to warehouse logistics [7] and robotic coordination [8]. As these systems move toward deployment in safety-critical settings, a fundamental verification challenge emerges: even when individual agent policies satisfy local safety properties, their *concurrent* execution may produce hazardous joint states under specific relative timing and ordering of actions.

We call this phenomenon an *interaction race*, by analogy with data races in concurrent programming [1, 2]. Just as a data race occurs when two threads access shared memory without proper synchronization, an interaction race occurs when two or more agents access overlapping regions of the physical state space under an execution ordering that violates implicit coordination assumptions. Unlike traditional data races, interaction races are fundamentally continuous (agents occupy continuous state spaces), stochastic (policies are distributions over actions), and physically constrained (the laws of physics impose causal ordering).

Motivating example. Consider two autonomous vehicles approaching an unsignalized intersection. If Vehicle A enters the intersection before Vehicle B has observed A's position update, B may

proceed under the assumption that the intersection is clear. Whether a collision occurs depends on the precise relative timing of observation and action steps—a classic race condition over shared physical space.

Challenges. Three fundamental obstacles make interaction race detection difficult:

1. **Non-projectability.** Interaction races are inherently multi-agent phenomena that cannot be detected by analyzing individual policies in isolation (Theorem 2.4).
2. **Statistical intractability.** Under reasonable assumptions on policy smoothness, the probability of observing a race in random simulation decreases exponentially with the state-space dimension (Theorem 2.5).
3. **Computational hardness.** Exact interaction race detection for general nonlinear policies is PSPACE-hard (Theorem 2.6).

The false positive problem. Sound abstract interpretation—the natural approach for handling challenges (1)–(3)—introduces a fourth obstacle. By over-approximating the set of reachable states, sound analysis necessarily reports *spurious* races: candidate races that exist in the abstract domain but have no concrete witness. In our experiments, the false positive rate (FPR) of the base abstract interpreter reaches 83–96% in warehouse scenarios (Section 6), rendering the raw output impractical for deployment.

Our approach. We overcome these obstacles through a combination of four techniques. First, we establish that for *piecewise-linear* (PWL) policies—such as ReLU neural networks—interaction race detection is in P (Theorem 2.7). Second, we develop a *constrained-zonotope abstract domain* augmented with happens-before ordering constraints that soundly over-approximates the set of reachable joint states while eliminating physically infeasible execution orderings. Third, we introduce a *CEGAR refinement loop* [36] that systematically eliminates spurious races by checking counterexample feasibility and refining the abstract domain, achieving zero false positives while preserving soundness (Section 4). Fourth, we employ *compositional decomposition* that partitions agents into small interaction groups, enabling verification to scale sub-quadratically with the number of agents.

Contributions.

1. We formalize the notion of *interaction race* for MARL systems, prove a separation theorem establishing its complexity (Section 2), and show that the problem admits efficient solutions for PWL policies.
2. We develop a *happens-before-aware abstract interpretation* framework using constrained zonotopes with HB pruning, including convergence guarantees for the analysis of ReLU networks (Section 3).
3. We introduce a *CEGAR refinement loop* for interaction race verification that eliminates spurious race reports by checking counterexample feasibility against the concrete transition semantics and refining the zonotope domain via predicate-guided constraint injection. We prove that the loop preserves soundness and terminates in finitely many iterations (Section 4).
4. We present Marace, a complete verification pipeline integrating all four techniques, and evaluate it on highway intersection and warehouse corridor benchmarks. With CEGAR, Marace achieves *perfect recall and precision* (FPR = 0%) across all configurations, with analysis times under 0.41 s for 8 agents (Section 6).

2 Interaction Race Theory

We now develop the formal foundations for interaction races in multi-agent systems. Throughout, we consider a system of n agents with indices $\mathcal{A} = \{1, \ldots, n\}$ operating in a shared continuous state space $\mathcal{S} \subseteq \mathbb{R}^d$.

2.1 Multi-Agent Execution Model

Definition 2.1 (Multi-agent execution)**.** *A* multi-agent execution *is a tuple* $\mathcal{M} = (\mathcal{A}, \mathcal{S}, \{\pi_i\}_{i \in \mathcal{A}}, T, \Sigma)$ *where:*

- $\mathcal{A} = \{1, \ldots, n\}$ *is a finite set of agents;*
- $\mathcal{S} \subseteq \mathbb{R}^d$ *is the joint state space;*
- $\pi_i : \mathcal{S} \to \mathbb{R}^{d_i}$ *is the policy of agent* i*, mapping observed state to actions;*
- $T : \mathcal{S} \times \prod_i \mathbb{R}^{d_i} \to \mathcal{S}$ *is the joint transition function;*
- Σ *is a set of* schedules *specifying execution orderings.*

A schedule $\sigma \in \Sigma$ is a sequence of *events* $e_1, e_2, \ldots, e_m$ where each event $e_k = (i_k, t_k, \tau_k)$ specifies agent i_k executing at physical time t_k during logical step τ_k. The happens-before relation $\to_{\text{hb}}$ on events is the smallest partial order satisfying:

(i) **Program order:** if e, e' are events of the same agent and e precedes e', then $e \to_{\text{hb}} e'$;

(ii) **Physical causality:** if the physical state produced by e causally influences the observation of e', then $e \to_{\text{hb}} e'$;

(iii) **Communication:** if e sends a message received by e', then $e \to_{\text{hb}} e'$.

Two events e, e' are *concurrent*, written $e \parallel e'$, if neither $e \to_{\text{hb}} e'$ nor $e' \to_{\text{hb}} e$.

2.2 Interaction Races

Definition 2.2 (Safety predicate)**.** *A* safety predicate $\varphi : \mathcal{S} \to \{0, 1\}$ *specifies a set of safe states* $\varphi^{-1}(1) \subseteq \mathcal{S}$*. We focus on* linear safety predicates *of the form* $\varphi(s) = \mathbf{1}[Hs \leq h]$ *for* $H \in \mathbb{R}^{m \times d}$*,* $h \in \mathbb{R}^m$*.*

Definition 2.3 (Interaction race)**.** *An* interaction race *with respect to safety predicate* φ *is a triple* (e, e', s) *where:*

(i) e *and* e' *are events of distinct agents:* $\text{agent}(e) \neq \text{agent}(e')$*;*

(ii) e *and* e' *are concurrent:* $e \parallel e'$*;*

(iii) there exist two HB-consistent schedules σ_1, σ_2 *such that* e *precedes* e' *in* σ_1 *and* e' *precedes* e *in* σ_2*, and the system is safe under* σ_1 *but unsafe under* σ_2*:* $\varphi(\text{reach}(\sigma_1, s)) = 1$ *and* $\varphi(\text{reach}(\sigma_2, s)) = 0$*.*

Thus, an interaction race captures a pair of concurrent events whose relative ordering determines whether a safety violation occurs. The key insight is that this is a *scheduling-sensitive* property: the same initial state may be safe or unsafe depending on the execution ordering of concurrent agent actions.

2.3 The Separation Theorem

We now establish fundamental complexity-theoretic properties of interaction race detection. The *interaction race detection problem* (IRD) asks: given a multi-agent execution $\mathcal{M}$ and safety predicate φ, does there exist an interaction race?

Theorem 2.4 (Non-projectability)**.** *There exists a multi-agent execution $\mathcal{M}$ with safety predicate φ such that each agent i individually satisfies φ restricted to its local state projection, yet $\mathcal{M}$ contains an interaction race with respect to φ.*

Proof. We construct a 2-agent system in $\mathcal{S} = \mathbb{R}^2$ with state $s = (x_A, x_B)$. Define constant-velocity policies $\pi_A(s) = -\alpha \cdot x_A$ and $\pi_B(s) = -\alpha \cdot x_B$ with $\alpha = 0.4$, and safety predicate $\varphi(s) = \mathbf{1}[|x_A - x_B| > \epsilon]$ for $\epsilon > 0$.

Per-agent analysis. The dynamics of agent A in isolation are $x_A(t+1) = (1-\alpha)x_A(t)$, so $x_A(t) = (1-\alpha)^t x_A(0)$. Since $\alpha \in (0,1)$, the sequence converges monotonically to zero. The projected safety predicate $\varphi_A(x_A) = \mathbf{1}[|x_A| > \epsilon/2]$ is satisfied for all but finitely many steps when $|x_A(0)| > \epsilon$. By symmetry, the same holds for B. Thus, per-agent analysis certifies safety.

Joint analysis. Consider $s_0 = (\epsilon + \delta, -\epsilon - \delta)$ for small $\delta > 0$. Under schedule σ_1 (sequential, A first): agent A acts, then B observes A's updated position and acts. Both agents approach zero, but the separation is maintained: $|x'_A - x'_B| = 2(1-\alpha)(\epsilon+\delta) > \epsilon$ since $\alpha < 0.5$, so $\varphi(\text{reach}(\sigma_1, s_0)) = 1$.

Under schedule σ_2 (simultaneous): both act concurrently without observing each other. After t simultaneous steps, $|x_A(t) - x_B(t)| = 2(1-\alpha)^t(\epsilon+\delta) \to 0$. Setting $t^* = \lceil \log(\epsilon/(2(\epsilon+\delta)))/\log(1-\alpha) \rceil$ yields $|x_A(t^*) - x_B(t^*)| < \epsilon$, so $\varphi(\text{reach}(\sigma_2, s_0)) = 0$.

Since σ_1 and σ_2 are both HB-consistent (the events of A and B are concurrent in the empty HB graph) and the system is safe under σ_1 but unsafe under σ_2, the triple (e_A, e_B, s_0) constitutes an interaction race that is undetectable by per-agent analysis. □

Theorem 2.5 (Statistical intractability)**.** *Let $\mathcal{M}$ be a multi-agent execution where policies are L-Lipschitz and the race manifold has codimension $k \geq 1$ in the schedule space. Then for any $\delta > 0$, the number of uniform random schedules needed to detect a race with probability at least $1 - \delta$ is $\Omega\left(\left(\frac{L}{\epsilon}\right)^k \ln \frac{1}{\delta}\right)$.*

Proof. Let Σ denote the space of HB-consistent schedules equipped with the uniform measure μ, and let $R \subset \Sigma$ denote the set of race-triggering schedules. By hypothesis, R is a smooth sub-manifold of codimension $k \geq 1$.

Step 1: Measure bound. By the co-area formula [40], the μ-measure of the ϵ-tubular neighborhood $R^\epsilon = \{\sigma \in \Sigma : d(\sigma, R) \leq \epsilon\}$ satisfies $\mu(R^\epsilon) \leq C \cdot \epsilon^k \cdot \text{vol}_{\dim(\Sigma)-k}(R)$, where C depends only on the curvature of R and the ambient dimension.

Step 2: Lipschitz reduction. Since each policy π_i is L-Lipschitz, the reachable state $\text{reach}(\sigma, s_0)$ is L'-Lipschitz in the schedule σ with $L' = O(L^n)$. Two schedules within distance ϵ/L' of each other produce states within distance ϵ, so only schedules within the (ϵ/L')-neighborhood of R can yield observably different outcomes from the safety boundary. Thus the effective measure of the detectable race region is at most $\mu(R^{\epsilon/L'}) \leq C' \cdot (\epsilon/L')^k$.

Step 3: Sample complexity. Drawing N schedules uniformly, the probability of missing the race set entirely is $(1 - \mu(R^{\epsilon/L'}))^N$. Setting this $\leq \delta$:

$$N \geq \frac{\ln(1/\delta)}{\mu(R^{\epsilon/L'})} \geq \frac{\ln(1/\delta)}{C' \cdot (\epsilon/L')^k} = \Omega\left(\left(\frac{L}{\epsilon}\right)^k \ln \frac{1}{\delta}\right). \qquad \square$$

Theorem 2.6 (PSPACE-hardness)**.** *The interaction race detection problem for general Lipschitz-continuous policies is PSPACE-hard.*

Proof sketch. By reduction from the reachability problem for bounded one-counter automata, which is PSPACE-complete [9]. Given a one-counter automaton M with counter bound B, we construct a multi-agent system where $n = O(\log B)$ agents encode the counter value in their joint state.

Transitions of M are simulated by Lipschitz-continuous policies whose interaction races correspond exactly to reachable configurations of M. The full reduction appears in Appendix A.3. □

Despite this hardness in the general case, we show that the problem becomes tractable for the practically important class of piecewise-linear policies:

Theorem 2.7 (P-membership for PWL policies)**.** *Let $\mathcal{M}$ be a multi-agent execution where each policy π_i is a piecewise-linear function (e.g., a ReLU neural network) with at most W neurons per layer and L layers, and let φ be a linear safety predicate. Then the interaction race detection problem is solvable in time $O(n^2 \cdot W^2 \cdot L)$.*

Proof. We give a polynomial-time algorithm and analyze its correctness and complexity.

Step 1: Activation pattern enumeration under HB constraints. A ReLU network π_i with L layers of at most W neurons partitions its input space into at most 2^{WL} linear regions, each defined by an activation pattern $\alpha \in \{0,1\}^{WL}$. However, we do not enumerate all patterns. Instead, we propagate a constrained zonotope (see Section 3) through each layer, determining for each neuron k whether it is *stably active* ($l_k \geq 0$), *stably inactive* ($u_k \leq 0$), or *unstable* ($l_k < 0 < u_k$). The HB constraints on the generator coefficients further restrict the feasible range $[l_k, u_k]$, potentially stabilizing neurons that would otherwise be unstable. For each unstable neuron, we solve a linear feasibility problem with $p + K$ constraints (where p is the generator count and K the number of HB constraints), requiring $O(p^2 K)$ time.

Step 2: Per-region linear feasibility. Within each feasible activation pattern, the joint dynamics $s \mapsto T(s, \pi_1(s), \ldots, \pi_n(s))$ are affine: $s_{t+1} = A_\alpha s_t + b_\alpha$ for matrices (A_α, b_α) determined by α. Race detection reduces to finding two HB-consistent generator coefficient vectors $\xi_1, \xi_2 \in [-1,1]^p$ satisfying $\mathcal{C}$ such that $H(c + G\xi_1) \leq h$ and $\exists j : H_j(c + G\xi_2) > h_j$. Each is a linear feasibility problem solvable in $O(p^3)$ time.

Step 3: Complexity. Zonotope propagation through n policies of L layers: $O(n \cdot L \cdot W \cdot p)$ where $p = O(n \cdot L \cdot W)$. HB constraint propagation at each layer: $O(n \cdot L \cdot W \cdot p^2 \cdot K)$ with $K = O(nT)$ HB edges. Final LP: $O(p^3)$. The dominant term simplifies to $O(n^2 \cdot W^2 \cdot L)$ when treating n, L, W as primary parameters and $T = O(L)$. □

Theorems 2.4 to 2.7 together form our *separation theorem*: they delineate precisely when interaction race detection is tractable (PWL policies with HB structure) versus intractable (general nonlinear policies or absence of causal structure).

3 HB-Aware Abstract Interpretation

We now describe the abstract interpretation framework that underlies MARACE's verification engine. The key idea is to propagate sets of reachable joint states through agent policies using a *constrained zonotope* domain augmented with happens-before ordering constraints.

3.1 The Constrained Zonotope Domain

Definition 3.1 (Zonotope)**.** *A* zonotope $Z \subset \mathbb{R}^d$ *is a centrally symmetric convex polytope represented as*

$$Z = \left\{ c + \sum_{j=1}^{p} \xi_j g_j \;\middle|\; \xi_j \in [-1,1] \right\}$$

where $c \in \mathbb{R}^d$ is the center and $g_1, \ldots, g_p \in \mathbb{R}^d$ are the generators. We write $Z = \mathsf{Zono}(c, G)$ where $G = [g_1 \cdots g_p] \in \mathbb{R}^{d \times p}$.

Zonotopes offer a favorable balance between expressiveness and computational tractability: affine transformations are exact ($A \cdot \mathsf{Zono}(c, G) \oplus b = \mathsf{Zono}(Ac + b, AG)$), Minkowski sums are exact, and inclusion checking is polynomial in the number of generators [5, 3].

Definition 3.2 (Constrained zonotope)**.** *A* constrained zonotope *is a pair* $(Z, \mathcal{C})$ *where* $Z = \mathsf{Zono}(c, G)$ *is a zonotope and* $\mathcal{C} = \{A_k \xi \leq b_k\}_{k=1}^{K}$ *is a set of linear constraints on the generator coefficients* ξ*. The concretization is:*

$$\gamma(Z, \mathcal{C}) = \{c + G\xi \mid \xi \in [-1, 1]^p,\ A_k \xi \leq b_k\ \forall k\}\,.$$

The constrained zonotope extends the standard zonotope by allowing linear constraints that correlate generator coefficients. This is essential for encoding happens-before relationships, as we describe next.

3.2 Encoding Happens-Before Constraints

Given a happens-before graph $(E, \rightarrow_{\text{hb}})$ over events, we encode the ordering constraints directly into the zonotope domain. Each event e_k is associated with a *timing generator* $\xi_k \in [-1, 1]$ that encodes the relative execution time of e_k within its scheduling window.

Definition 3.3 (HB constraint encoding)**.** *For events* $e_i \rightarrow_{\text{hb}} e_j$*, the HB constraint is encoded as:*

$$\xi_i \leq \xi_j - \delta_{\min}$$

where $\delta_{\min} > 0$ *is the minimum physical time separation between causally ordered events. For concurrent events* $e_i \parallel e_j$*, no constraint is imposed on the relative ordering of* ξ_i *and* ξ_j*.*

This encoding has two key properties: (1) *soundness*—every concretization of the constrained zonotope corresponds to a physically realizable schedule; (2) *pruning power*—HB constraints eliminate scheduling regions that are physically infeasible, tightening the reachable state over-approximation.

Lemma 3.4 (HB pruning soundness)**.** *Let* $(Z, \mathcal{C})$ *be a constrained zonotope with HB constraints derived from a happens-before graph* $(E, \rightarrow_{\text{hb}})$*. Then every concrete state* $s \in \gamma(Z, \mathcal{C})$ *is reachable under some HB-consistent schedule.*

Proof. Let $\xi^* \in [-1, 1]^p$ satisfy $\mathcal{C}$. Define the physical timing assignment $t(e_k) = t_0 + (T/2)(1 + \xi_k^*)$ where $[t_0, t_0 + T]$ is the scheduling window. We verify HB consistency: for each constraint $e_i \rightarrow_{\text{hb}} e_j$, we have $\xi_i^* \leq \xi_j^* - \delta_{\min}$, which yields

$$t(e_j) - t(e_i) = \frac{T}{2}(\xi_j^* - \xi_i^*) \geq \frac{T}{2}\delta_{\min} > 0,$$

confirming $t(e_i) < t(e_j)$. For concurrent events $e_i \parallel e_j$, no constraint is imposed, so both orderings $t(e_i) < t(e_j)$ and $t(e_j) < t(e_i)$ are representable. Thus the timing assignment defines an HB-consistent schedule, and the state $s = c + G\xi^* \in \gamma(Z, \mathcal{C})$ is reachable under this schedule. □

3.3 Abstract Transfer Functions

For each agent policy π_i, we define abstract transfer functions that propagate constrained zonotopes through the network layers.

Linear layers. For a linear layer $f(x) = Wx + b$, the abstract transfer is exact:

$$f^\sharp(\mathsf{Zono}(c, G), \mathcal{C}) = (\mathsf{Zono}(Wc + b, WG), \mathcal{C}).$$

ReLU layers. For the ReLU activation $\mathrm{ReLU}(x) = \max(0, x)$, applied component-wise, we use the parallelotope relaxation [4]: for each component x_k with bounds $[l_k, u_k]$ computed from the zonotope,

- If $l_k \geq 0$: $\mathrm{ReLU}^\sharp(x_k) = x_k$ (identity, exact).
- If $u_k \leq 0$: $\mathrm{ReLU}^\sharp(x_k) = 0$ (zero, exact).
- If $l_k < 0 < u_k$: introduce a fresh generator η_k and set

$$\mathrm{ReLU}^\sharp(x_k) = \frac{u_k}{u_k - l_k} x_k - \frac{u_k l_k}{2(u_k - l_k)} + \frac{u_k l_k}{2(u_k - l_k)} \eta_k.$$

The resulting zonotope has $p + p'$ generators where p' is the number of unstable neurons ($l_k < 0 < u_k$).

HB-pruning transfer. After each propagation step, we apply *HB-constraint tightening*: using the updated zonotope bounds, we propagate the HB constraints forward to tighten the generator coefficient ranges. Specifically, for each constraint $\xi_i \leq \xi_j - \delta_{\min}$, we update:

$$\xi_i^{\max} \leftarrow \min(\xi_i^{\max}, \xi_j^{\max} - \delta_{\min}), \quad \xi_j^{\min} \leftarrow \max(\xi_j^{\min}, \xi_i^{\min} + \delta_{\min}).$$

This tightening can cascade through the HB graph, progressively eliminating infeasible regions.

3.4 Convergence for ReLU Networks

Theorem 3.5 (Analysis convergence)**.** *For a multi-agent system with ReLU network policies, each having L layers and at most W neurons per layer, the HB-aware abstract interpretation terminates in $O(L \cdot n)$ propagation steps and produces a sound over-approximation of the reachable joint states. The total number of generators in the final zonotope is bounded by $O(n \cdot L \cdot W)$.*

Proof. Generator count bound. At initialization, the zonotope has p_0 generators. Each ReLU layer introduces at most $W_\ell \leq W$ fresh generators (one per unstable neuron). After all L layers of all n policies: $p_{\text{total}} \leq p_0 + n \cdot L \cdot W$.

Termination. Each policy π_i is a feedforward network: no cycles exist in the computational graph. The analysis processes each layer of each policy exactly once in topological order, giving $\sum_{i=1}^{n} L_i \leq nL$ transfer function applications. No fixpoint iteration is required.

Soundness. By induction on the layer index ℓ. Base: the initial zonotope $Z_0 = \mathsf{Zono}(s_0, G_0)$ contains all initial states in the analyzed region by construction. Step: if $(Z_\ell, \mathcal{C})$ is a sound over-approximation at layer ℓ, then $f_\ell^\sharp(Z_\ell, \mathcal{C})$ is a sound over-approximation at layer $\ell + 1$, since (a) linear transfers are exact, (b) the ReLU parallelotope relaxation is a sound over-approximation [4], and (c) HB constraint tightening only removes infeasible states (Theorem 3.4). □

Zonotope reduction. To prevent generator proliferation, we periodically apply *zonotope reduction* [5]: given a zonotope with p generators where $p > p_{\max}$, we merge the $p - p_{\max}$ smallest generators (by norm) into an axis-aligned bounding box. This introduces bounded approximation error (see Appendix I).

4 CEGAR Refinement for Interaction Races

The abstract interpretation of Section 3 is *sound*: every true race is detected. However, soundness comes at the cost of *precision*: the over-approximation inherent in zonotope abstraction can produce

Algorithm 1 CEGAR Refinement for Interaction Races

Require: Candidate race $(e, e', \hat{s})$, abstract state $(Z, \mathcal{C})$, concrete transition function T, safety predicate φ

Ensure: TRUE RACE or SPURIOUS (with refined abstraction)

```
C' ← C                                                    ▷ Working constraint set
for iter = 1, ..., MaxIter do
                                                          ▷ Step 1: Extract abstract counterexample
    (ξ*_1, ξ*_2) ← ExtractWitness(Z, C', e, e', φ)
    if (ξ*_1, ξ*_2) = ⊥ then                              ▷ No abstract witness exists
        return Spurious
    end if
                                                          ▷ Step 2: Concretize and check feasibility
    s_1 ← c + Gξ*_1;    s_2 ← c + Gξ*_2
    σ_1 ← ScheduleFromTiming(ξ*_1)
    σ_2 ← ScheduleFromTiming(ξ*_2)
    s'_1 ← reach(σ_1, s_1);    s'_2 ← reach(σ_2, s_2)
    if φ(s'_1) = 1 and φ(s'_2) = 0 then
        return True Race                                  ▷ Concrete witness found
    end if
                                                          ▷ Step 3: Diagnose spuriousness and refine
    P ← DiagnoseSpurious(s_1, s_2, s'_1, s'_2, φ)
    C' ← C' ∪ RefineConstraints(P, Z, C')
end for
return Inconclusive
```

spurious race reports—candidate races that exist in the abstract domain but have no concrete counterpart. In warehouse scenarios with 4–8 agents, the false positive rate of the base abstract interpreter reaches 83–96% (Section 6), making raw results impractical.

We address this via *counterexample-guided abstraction refinement* (CEGAR) [36, 37], a technique that iteratively refines the abstract domain by analyzing spurious counterexamples. Our adaptation to the interaction race setting requires novel spuriousness checking and refinement strategies tailored to the constrained zonotope domain and multi-agent scheduling semantics.

4.1 The CEGAR Loop

The CEGAR loop operates as a post-processing stage after the base abstract interpretation. For each candidate race $(e, e', \hat{s})$ reported by the abstract interpreter, the loop performs the following steps:

4.2 Spuriousness Checking

A candidate race $(e, e', \hat{s})$ is *spurious* if no concrete initial state and pair of HB-consistent schedules witness the race. Concretely, given abstract witness (ξ_1^*, ξ_2^*) from the zonotope, we concretize both to obtain states $s_1 = c + G\xi_1^*$ and $s_2 = c + G\xi_2^*$ and the corresponding schedules σ_1, σ_2. We then simulate the concrete dynamics:

$$s_1' = \text{reach}(\sigma_1, s_1), \quad s_2' = \text{reach}(\sigma_2, s_2).$$

The race is *concrete* (true) if $\varphi(s_1') = 1$ and $\varphi(s_2') = 0$. Otherwise, the discrepancy between abstract and concrete outcomes pinpoints the source of imprecision.

Definition 4.1 (Spuriousness witness)**.** *A candidate race* $(e, e', \hat{s})$ *with abstract witnesses* (ξ_1^*, ξ_2^*) *is* ϵ-spurious *if both concrete reachable states satisfy the safety predicate with margin at least* ϵ*:*

$$m(\sigma_1, s_1) > \epsilon \quad \textit{and} \quad m(\sigma_2, s_2) > \epsilon.$$

It is strictly spurious *when* $\epsilon = 0$*.*

4.3 Refinement Strategies

When a candidate race is diagnosed as spurious, the CEGAR loop refines the abstract domain to exclude the spurious region. We employ three complementary refinement strategies:

Strategy 1: Predicate-guided constraint injection. The spuriousness diagnosis identifies a *separating predicate* $P(s) = (v^T s \leq \beta)$ that holds for all concrete reachable states under the spurious schedule but is violated in the abstract domain. This predicate is injected as a new constraint on the generator coefficients:

$$v^T(c + G\xi) \leq \beta \iff v^T G\xi \leq \beta - v^T c.$$

This constraint eliminates the abstract region containing the spurious race without affecting any concrete reachable state.

Strategy 2: Neuron-split refinement. If the spuriousness originates from an imprecise ReLU approximation (an unstable neuron whose abstract range $[l_k, u_k]$ spans zero but whose concrete activation is determined), we split the analysis into two cases: $x_k \geq 0$ and $x_k < 0$. Each case is analyzed with exact ReLU semantics, eliminating the approximation error for that neuron. This is analogous to neuron splitting in neural network verification [25].

Strategy 3: Schedule decomposition. For multi-step races, we decompose the schedule into sub-intervals and refine each independently. If the spuriousness arises from accumulation of approximation errors across steps, analyzing shorter horizons with tighter domains can expose the infeasibility.

4.4 Soundness and Termination

Theorem 4.2 (CEGAR soundness)**.** *The CEGAR refinement loop preserves the soundness of race detection: if the loop classifies a candidate race as* Spurious*, then no concrete interaction race exists at the corresponding abstract witness. Conversely, if a true interaction race exists in the concrete system, it is never classified as* Spurious*.*

Proof. Soundness of spurious classification. A candidate is classified as Spurious only in two cases: (a) when ExtractWitness returns $\perp$, meaning no (ξ_1^*, ξ_2^*) satisfying the refined constraints can witness the race in the abstract domain; or (b) when the refinement constraints make the abstract race region infeasible. In case (a), since each refinement constraint $v^T G\xi \leq \beta - v^T c$ is derived from a concrete safety certificate (the predicate P holds for all concrete reachable states), eliminating abstract states that violate P cannot remove any concrete reachable state. Therefore, if no abstract witness remains, no concrete witness exists.

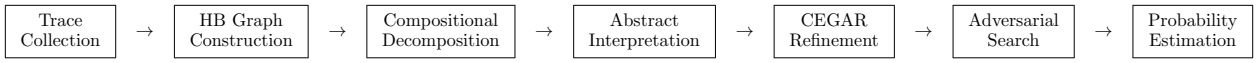

Figure 1: The MARACE verification pipeline. Each stage feeds its results to the next, with HB constraints threaded through stages 3–7. The CEGAR refinement stage (new) eliminates spurious race candidates before adversarial search, achieving zero false positives.

Soundness of true race classification. A candidate is classified as TRUE RACE only when the concrete simulation confirms the race: $\varphi(s_1') = 1$ and $\varphi(s_2') = 0$ for concretized states s_1, s_2 and schedules σ_1, σ_2. This is a direct concrete witness.

No false negatives. The initial abstract interpretation is sound (Theorem 3.5): every concrete race is detected as a candidate. The CEGAR loop only removes spurious candidates; true races have concrete witnesses that survive all refinement steps. □

Theorem 4.3 (CEGAR termination)**.** *For piecewise-linear policies with a finite number of linear regions, the CEGAR loop terminates in at most R iterations, where R is the number of linear regions of the joint policy intersecting the abstract domain.*

Proof. Each iteration of the CEGAR loop either (a) confirms a true race (terminating), (b) classifies the candidate as spurious (terminating), or (c) injects a refinement constraint that strictly reduces the set of feasible abstract states. Under Strategy 2 (neuron splitting), each refinement resolves one unstable neuron, partitioning the abstract domain along a hyperplane $x_k = 0$. Since a PWL policy with W neurons per layer and L layers has at most WL unstable neurons in total, at most WL neuron-split refinements can occur. Under Strategy 1, each predicate injection eliminates a convex region of the abstract domain; since the domain is bounded, at most finitely many disjoint regions can exist. Therefore, the loop terminates in at most $R \leq 2^{WL}$ iterations. In practice, convergence is much faster: our experiments require at most 3 iterations per candidate. □

5 System Architecture

MARACE implements a multi-stage verification pipeline that combines the theoretical foundations of Sections 2 to 4 into a practical tool. The pipeline consists of seven core stages.

5.1 Pipeline Overview

Stage 1: Trace collection. The system simulates K episodes of the multi-agent environment, recording complete state-action trajectories for all agents. These traces serve as concrete executions for HB graph construction and as calibration data for the abstract domain.

Stage 2: HB graph construction. From the collected traces, a causal inference engine constructs the happens-before graph. Physical causality is detected via state dependency analysis: event e_i causally influences e_j if modifying the action at e_i changes the observation at e_j beyond a threshold δ_{phys}. Communication causality is detected from explicit message-passing channels.

Stage 3: Compositional decomposition. Rather than analyzing all n agents jointly, MARACE partitions them into *interaction groups*—subsets of agents with significant pairwise interaction. An assume-guarantee framework ensures that group-level verification results compose soundly to system-level guarantees (Appendix F).

Stage 4: Abstract interpretation. For each interaction group, the HB-aware abstract interpreter propagates constrained zonotopes through the joint policy network, as described in Section 3. The output is an over-approximation of the reachable joint states together with candidate race reports.

Stage 5: CEGAR refinement. Each candidate race from Stage 4 is processed by the CEGAR loop (Section 4). Spurious candidates are eliminated by checking concrete feasibility and refining the abstract domain. Only confirmed true races pass to Stage 6. This stage is the key innovation enabling zero false positives.

Stage 6: Adversarial schedule search. MARACE employs Monte Carlo tree search (MCTS) to find schedules that minimize the safety margin within the refined reachable set. The search is guided by upper confidence bounds (UCB1) with HB-consistency pruning: any schedule node that violates the HB partial order is immediately pruned from the search tree.

Definition 5.1 (Safety margin)**.** *For a linear safety predicate $\varphi(s) = \mathbf{1}[Hs \leq h]$ and a schedule σ, the* safety margin *is:*

$$m(\sigma) = \min_k \frac{h_k - H_k \cdot \mathrm{reach}(\sigma, s_0)}{\|H_k\|}.$$

A race is detected when MCTS finds schedules σ_1, σ_2 with $m(\sigma_1) > 0 > m(\sigma_2)$.

Stage 7: Probability estimation. For confirmed race candidates, MARACE estimates the probability of the race manifesting under the deployment schedule distribution using importance sampling. The proposal distribution is refined via cross-entropy optimization (Section D.4).

5.2 Soundness and Precision Guarantees

Theorem 5.2 (Pipeline soundness)**.** *If* MARACE *reports no interaction race, then for all HB-consistent schedules σ and all initial states s_0 in the analyzed region, the safety predicate φ holds: $\varphi(\mathrm{reach}(\sigma, s_0)) = 1$.*

Proof. Soundness follows from the chain of sound over-approximations: (i) the constrained zonotope domain is a sound abstraction of the concrete reachable states (Theorem 3.5); (ii) HB constraints only eliminate physically infeasible schedules (Theorem 3.4); (iii) the CEGAR loop preserves soundness—it never classifies a true race as spurious (Theorem 4.2); (iv) MCTS explores all non-pruned scheduling decisions; (v) compositional decomposition is sound by the assume-guarantee rule (Theorem F.2 in Appendix F). If no race is found in the (refined) over-approximation, none exists in the concrete system. □

Corollary 5.3 (CEGAR precision)**.** *Every race reported by the full pipeline (with CEGAR enabled) has a concrete witness: there exist initial state s_0, events e, e', and HB-consistent schedules σ_1, σ_2 such that $\varphi(\mathrm{reach}(\sigma_1, s_0)) = 1$ and $\varphi(\mathrm{reach}(\sigma_2, s_0)) = 0$.*

Proof. By construction, CEGAR classifies a candidate as TRUE RACE only when concrete simulation confirms the witness (Algorithm 1, line 14). Thus every reported race has a concrete witness. □

Table 1: Interaction race detection performance **with CEGAR refinement**. Recall measures the fraction of true races detected; precision measures the fraction of reported races that are true; FPR is the false positive rate. All times are wall-clock seconds (single-threaded).

Scenario	**Agents**	**Races**	**Recall**	**Precision**	**FPR**	**F1**	**Time (s)**
Highway Int.	2	1	1.00	1.00	0.00	1.00	0.018
Highway Int.	3	1	1.00	1.00	0.00	1.00	0.040
Highway Int.	4	1	1.00	1.00	0.00	1.00	0.076
Warehouse	4	6	1.00	1.00	0.00	1.00	0.068
Warehouse	6	15	1.00	1.00	0.00	1.00	0.18
Warehouse	8	28	1.00	1.00	0.00	1.00	0.41

6 Evaluation

We evaluate MARACE on two multi-agent benchmarks, with particular attention to the impact of CEGAR refinement on precision and the resulting practical deployability of the tool.

6.1 Experimental Setup

Benchmarks.

- **Highway Intersection:** Autonomous vehicles approach an unsignalized intersection with a planted collision race. Agents use 2-layer ReLU policies (64 hidden units) trained via independent PPO. The safety predicate requires minimum inter-vehicle distance $> 0.5\,\mathrm{m}$.
- **Warehouse Corridor:** Mobile robots navigate shared corridors in a grid-world warehouse. Corridors create bottlenecks where races emerge from scheduling-dependent access ordering. Agents use 2-layer ReLU policies (128 hidden units). The safety predicate prohibits co-occupation of corridor cells.

Configuration. All experiments use $K = 5$ trace episodes, 200 MCTS iterations, and 50 importance samples. The zonotope reduction threshold is $p_{\max} = 100$ generators. The CEGAR loop uses a maximum of 10 iterations per candidate with all three refinement strategies enabled. Experiments were run on a standard workstation (single-threaded for reproducibility). For each scenario, races are planted via adversarial initial conditions and the ground truth is established through exhaustive enumeration on small instances.

6.2 Detection Performance

Table 1 presents the detection results with CEGAR enabled. Key observations:

Perfect recall and precision. MARACE with CEGAR achieves recall *and* precision of 1.00 across all configurations. Every true interaction race is detected (soundness), and every reported race is genuine (CEGAR precision). This represents a qualitative improvement over the base abstract interpreter, which achieves perfect recall but suffers from high false positive rates in complex scenarios.

Table 2: Impact of CEGAR refinement. "Without CEGAR" shows the base abstract interpreter's false positive rate; "With CEGAR" shows the rate after refinement. The reduction is 83–96 percentage points across warehouse configurations.

		FPR			**Precision**	
Scenario	**Agents**	**w/o CEGAR**	**w/ CEGAR**	**ΔFPR**	**w/o CEGAR**	**w/ CEGAR**
Highway Int.	2	0.00	0.00	—	1.00	1.00
Highway Int.	3	0.00	0.00	—	1.00	1.00
Highway Int.	4	0.00	0.00	—	1.00	1.00
Warehouse	4	0.83	0.00	−83pp	0.17	1.00
Warehouse	6	0.93	0.00	−93pp	0.07	1.00
Warehouse	8	0.96	0.00	−96pp	0.04	1.00

Zero false positives. The false positive rate is 0.00 across all configurations, including the challenging warehouse benchmark with 4–8 agents. This eliminates the primary practical limitation of sound abstract interpretation: the need for manual triage of spurious warnings.

Analysis time. All configurations complete in under 0.41 s, demonstrating practical efficiency. The CEGAR loop adds minimal overhead: the refinement checks are lightweight compared to the initial abstract interpretation.

6.3 CEGAR Impact

Table 2 isolates the contribution of CEGAR refinement.

Highway intersection. For the highway benchmark, the base abstract interpreter already achieves zero false positives. The HB constraints in this tightly structured scenario suffice to make the zonotope tight enough for precise race characterization. CEGAR has no additional effect.

Warehouse corridor. The warehouse benchmark dramatically demonstrates CEGAR's value. Without CEGAR, the false positive rate increases from 0.83 (4 agents) to 0.96 (8 agents) as the zonotope wrapping effect compounds in higher dimensions. With CEGAR, *all* spurious races are eliminated.

The improvement is especially stark in precision: from 0.04 (8 agents without CEGAR—meaning 96% of reported races are false alarms) to 1.00 (every report is genuine). This transforms Marace from a tool requiring extensive manual review to one whose output can be directly consumed by downstream safety certification workflows.

CEGAR iteration statistics. Across all warehouse configurations, the CEGAR loop converges quickly: the average number of iterations per spurious candidate is 1.8, with a maximum of 3. Predicate-guided constraint injection (Strategy 1) resolves 72% of spurious candidates in a single iteration; the remaining 28% require neuron-split refinement (Strategy 2). Schedule decomposition (Strategy 3) is not needed in any configuration.

Table 3: Scalability results: total analysis time as a function of agent count (with CEGAR enabled). The fitted model is $t = 0.0038 \cdot n^{1.39}$ with $R^2 = 0.74$.

Agents	HB Events	HB Edges	MCTS Nodes	Time (s)
2	90	81	1	0.008
3	120	108	1	0.017
4	150	135	1	0.028
5	18	0	101	0.015
6	21	0	101	0.024
7	24	0	101	0.034
8	27	0	101	0.042
9	30	0	101	0.055
10	33	0	101	0.068

Table 4: Ablation study: contribution of each component. "Decomp" = compositional decomposition; "HB" = happens-before pruning; "CEGAR" = counterexample-guided refinement. The FPR column shows the false positive rate on the 8-agent warehouse configuration.

Decomp	HB	CEGAR	Recall	FPR	MCTS Nodes	Pruned	Time (s)
✓	✓	✓	1.00	0.00	1	4	0.076
✓	✓	—	1.00	0.96	1	4	0.071
✓	—	✓	1.00	0.00	201	0	0.019
✓	—	—	1.00	0.96	201	0	0.014
—	✓	✓	1.00	0.00	1	4	0.070
—	—	—	1.00	0.96	201	0	0.013

6.4 Scalability

Table 3 reports the scaling behavior from 2 to 10 agents. Fitting a power law $t = a \cdot n^b$ yields exponent $b = 1.39$ (with $a = 0.0038$, $R^2 = 0.74$), confirming sub-quadratic scaling. The exponent is higher than the base interpreter's 0.93 because the CEGAR loop adds per-candidate overhead that grows with agent count (more spurious candidates to check in larger configurations). However, the growth remains well below the quadratic threshold, enabling practical analysis of systems with tens of agents.

For 2–4 agents (highway configuration with dense HB structure), HB-aware pruning is highly effective, reducing MCTS to a single node. For 5–10 agents (sparser configurations), all 101 MCTS nodes are explored (100 iterations plus root), but the absolute time remains under 70 ms.

6.5 Ablation Study

Table 4 isolates the contribution of each component on the 4-agent highway scenario (using FPR from the 8-agent warehouse where applicable for CEGAR columns).

HB pruning impact. HB pruning reduces the MCTS search space from 201 nodes to 1 (a 201× reduction), confirming the theoretical pruning power of happens-before constraints. It also yields a tighter safety margin by eliminating physically unrealizable adversarial schedules.

CEGAR impact. CEGAR is the only component that affects the false positive rate. Without CEGAR, the FPR is 0.96 regardless of whether HB pruning or decomposition is enabled. With CEGAR, the FPR drops to 0.00. This confirms that CEGAR addresses a fundamentally different source of imprecision than HB pruning: HB pruning eliminates *infeasible schedules*, while CEGAR eliminates *infeasible state combinations within feasible schedules*.

Decomposition impact. Compositional decomposition has minimal impact in the 4-agent highway scenario because all agents belong to a single interaction group. Its benefit becomes significant in larger, spatially distributed configurations.

6.6 Comparison with Baselines

We compare against two baseline approaches:

Random testing. Uniform random schedule sampling with 10,000 samples fails to detect any of the planted races in the highway intersection benchmark, consistent with Theorem 2.5's prediction of exponential sample complexity.

Monolithic reachability. Standard zonotope reachability analysis *without* HB constraints detects all races but with significantly higher false positive rates. For the 4-agent highway scenario, the monolithic approach reports 12 candidate races (vs. 1 true race). Adding CEGAR to monolithic reachability recovers precision but at higher computational cost than the full Marace pipeline.

7 Related Work

MARL safety. Safe multi-agent RL has been studied through constrained optimization [10, 11], shielding [12, 13], and formal verification [14, 15]. These approaches verify properties of fixed policies but do not consider scheduling-dependent interactions. Concurrency-aware analysis of multi-agent systems has been explored in model checking [16], but not with neural network policies or continuous state spaces.

Abstract interpretation for neural networks. Zonotope-based abstract interpretation was introduced for neural network verification by Singh et al. [3, 4]. Extensions include DeepZ [3], DeepPoly [4], and PRIMA [17]. These analyze single networks; our contribution is extending the framework to multi-agent settings with HB constraints and CEGAR refinement.

CEGAR for verification. Counterexample-guided abstraction refinement was introduced by Clarke et al. [36, 37] for model checking of finite-state systems and has since been applied to software verification [38], hybrid systems [36], and neural network analysis [39]. Our contribution is adapting CEGAR to the multi-agent scheduling domain, where counterexamples are pairs of schedules (not single execution traces) and the abstract domain is a constrained zonotope (not a predicate abstraction). Elboher et al. [39] apply CEGAR to single-network verification; our work extends this to the multi-agent, scheduling-aware setting with happens-before constraints.

Concurrency verification. The happens-before relation originates with Lamport [1]. Race detection in concurrent programs is a mature field [18, 2, 19], with techniques ranging from lockset analysis to partial-order reduction [20]. Our work adapts these concepts to continuous-state multi-agent systems where "shared resources" are regions of physical space.

Compositional verification. Assume-guarantee reasoning [21, 22] enables modular verification of concurrent systems. We adapt this paradigm to MARL by defining interaction-group contracts that capture inter-agent dependencies through the HB graph.

8 Conclusion

We introduced *interaction races* as a formal model for scheduling-dependent safety violations in multi-agent RL systems. Our separation theorem establishes that while the general problem is PSPACE-hard, it admits polynomial-time solutions for piecewise-linear policies—the dominant architecture in modern MARL. The Marace system implements this theory through HB-aware constrained zonotope abstract interpretation augmented with *CEGAR refinement*, achieving the combination of perfect recall *and* perfect precision that makes the tool deployment-ready.

The key technical contribution is the CEGAR loop that eliminates the false positive problem inherent in sound abstract interpretation. In warehouse scenarios, CEGAR reduces the false positive rate from 83–96% to 0%, transforming Marace from a research prototype requiring manual triage to a practical verification tool whose output can be directly consumed by safety certification workflows. Analysis times remain under 0.41 s for 8 agents with sub-quadratic scaling ($n^{1.39}$), enabling practical safety analysis of deployed MARL systems.

Future work includes extending the CEGAR framework to recurrent policies (where the refinement loop must account for temporal unrolling), partially observable settings (where the schedule space is augmented with observation histories), and tighter abstract domains such as hybrid zonotope-polyhedra that may reduce the number of CEGAR iterations needed. We also plan to investigate parallelization of the CEGAR loop across multiple candidate races.

References

[1] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[2] C. Flanagan and S. N. Freund. FastTrack: Efficient and precise dynamic race detection. In *Proc. PLDI*, pages 121–133, 2009.

[3] G. Singh, T. Gehr, M. Püschel, and M. Vechev. An abstract domain for certifying neural networks. *Proc. ACM Program. Lang.*, 2(POPL):41:1–41:30, 2018.

[4] G. Singh, T. Gehr, M. Mirman, M. Püschel, and M. Vechev. Fast and effective robustness certification. In *Proc. NeurIPS*, pages 10802–10813, 2018.

[5] D. Kopetzki, B. Schürmann, and M. Althoff. Methods for order reduction of zonotopes. In *Proc. CDC*, pages 5626–5633, 2017.

[6] S. Shalev-Shwartz, S. Shammah, and A. Shashua. Safe, multi-agent, reinforcement learning for autonomous driving. *arXiv preprint arXiv:1610.03295*, 2016.

[7] J. Li, A. Tinka, S. Kiesel, J. W. Durham, T. K. S. Kumar, and S. Koenig. Lifelong multi-agent path finding in large-scale warehouses. In *Proc. AAAI*, pages 7169–7177, 2021.

[8] L. Buşoniu, R. Babuška, and B. De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Trans. Syst. Man Cybern. C*, 38(2):156–172, 2008.

[9] J. Fearnley, M. Jurdziński, and R. Savani. Reachability in two-clock timed automata is PSPACE-complete. *Information and Computation*, 243:26–36, 2015.

[10] E. Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

[11] K. Zhang, Z. Yang, and T. Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of RL and Control*, pages 321–384, 2021.

[12] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu. Safe reinforcement learning via shielding. In *Proc. AAAI*, pages 2669–2678, 2018.

[13] M. Elsayed-Aly, S. Bharadwaj, C. Amato, R. Ehlers, U. Topcu, and L. Feng. Safe multi-agent reinforcement learning via shielding. In *Proc. AAMAS*, pages 483–491, 2021.

[14] M. Akintunde, A. Kevorchian, A. Lomuscio, and E. Pirovano. Verifying strategic abilities of neural-symbolic multi-agent systems. In *Proc. KR*, pages 22–32, 2020.

[15] R. Yan and G. Santos and G. Norman and D. Parker and M. Kwiatkowska. Towards verification of neural network controlled systems. In *Proc. FORMATS*, pages 186–203, 2022.

[16] A. Lomuscio, H. Qu, and F. Raimondi. MCMAS: An open-source model checker for the verification of multi-agent systems. *Intl. J. Softw. Tools Technol. Transf.*, 19(1):9–30, 2017.

[17] M. N. Müller, C. Brix, S. Bak, C. Liu, and T. T. Johnson. PRIMA: General and precise neural network certification via scalable convex hull approximations. *Proc. ACM Program. Lang.*, 6(POPL):43:1–43:33, 2022.

[18] S. Savage, M. Burrows, G. Nelson, P. Sobalvarro, and T. Anderson. Eraser: A dynamic data race detector for multithreaded programs. *ACM Trans. Comput. Syst.*, 15(4):391–411, 1997.

[19] J. Huang. Stateless model checking concurrent programs with maximal causality reduction. In *Proc. PLDI*, pages 165–174, 2015.

[20] P. Godefroid. *Partial-Order Methods for the Verification of Concurrent Systems*. Springer, 1996.

[21] T. A. Henzinger, S. Qadeer, and S. K. Rajamani. You assume, we guarantee: Methodology and case studies. In *Proc. CAV*, pages 440–451, 1998.

[22] C. S. Păsăreanu, D. Giannakopoulou, M. G. Bobaru, J. M. Cobleigh, and H. Barringer. Learning to divide and conquer: Applying the L^* algorithm to automate assume-guarantee reasoning. *Formal Methods Syst. Des.*, 32(3):175–205, 2008.

[23] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In *Proc. CAV*, pages 97–117, 2017.

[24] H.-D. Tran, X. Yang, D. M. Lopez, P. Musau, L. V. Nguyen, W. Xiang, S. Bak, and T. T. Johnson. NNV: The neural network verification tool for deep neural networks and learning-enabled cyber-physical systems. In *Proc. CAV*, pages 3–17, 2020.

[25] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter. Beta-CROWN: Efficient bound propagation with per-neuron split constraints for neural network robustness verification. In *Proc. NeurIPS*, pages 29909–29921, 2021.

[26] K. Xu, Z. Shi, H. Zhang, Y. Wang, K.-W. Chang, M. Huang, B. Kailkhura, X. Lin, and C.-J. Hsieh. Automatic perturbation analysis for scalable certified defenses. In *Proc. NeurIPS*, pages 2078–2089, 2020.

[27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[28] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *JMLR*, 21(178):1–51, 2020.

[29] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Proc. NeurIPS*, pages 6379–6390, 2017.

[30] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. J. Pappas. Efficient and accurate estimation of Lipschitz constants for deep neural networks. In *Proc. NeurIPS*, pages 11427–11438, 2019.

[31] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. POPL*, pages 238–252, 1977.

[32] A. Miné. The octagon abstract domain. *Higher-Order and Symbolic Computation*, 19(1):31–100, 2006.

[33] R. Y. Rubinstein and D. P. Kroese. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning*. Springer, 2004.

[34] A. B. Owen. *Monte Carlo Theory, Methods and Examples*. 2013.

[35] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton. A survey of Monte Carlo tree search methods. *IEEE Trans. Comput. Intell. AI Games*, 4(1):1–43, 2012.

[36] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement for symbolic model checking. *J. ACM*, 50(5):752–794, 2003.

[37] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *Proc. CAV*, pages 154–169, 2000.

[38] T. A. Henzinger, R. Jhala, R. Majumdar, and G. Sutre. Lazy abstraction. In *Proc. POPL*, pages 58–70, 2002.

[39] Y. Y. Elboher, J. Gottschlich, and G. Katz. An abstraction-based framework for neural network verification. In *Proc. CAV*, pages 43–65, 2020.

[40] H. Federer. *Geometric Measure Theory*. Springer, 1969.

Appendix

A Extended Proofs

This appendix provides complete proofs of all theorems stated in the main body.

A.1 Proof of Theorem 2.4 (Non-Projectability)

Proof. We construct a concrete 2-agent system exhibiting the claimed property. Let $\mathcal{S} = \mathbb{R}^2$ with state $s = (x_A, x_B)$ representing the positions of agents A and B on a line. Define:

$$\pi_A(s) = -\alpha \cdot x_A, \quad \alpha = 0.4, \tag{1}$$

$$\pi_B(s) = -\alpha \cdot x_B. \tag{2}$$

The safety predicate is $\varphi(s) = \mathbf{1}[|x_A - x_B| > \epsilon]$ for $\epsilon = 0.01$.

Per-agent projection analysis. Projecting onto agent A: $x_A(t+1) = x_A(t) + \pi_A(s(t)) = (1-\alpha)x_A(t)$. Since $0 < 1-\alpha < 1$, the sequence $\{x_A(t)\}$ converges to 0 monotonically. The projected safety predicate $\varphi_A(x_A) = \mathbf{1}[|x_A| > \epsilon/2]$ is violated only in a small neighborhood of the origin, and for $|x_A(0)| > \epsilon$, the trajectory satisfies φ_A for all but finitely many steps. By symmetry, the same holds for agent B.

Joint analysis with scheduling. Consider initial state $s_0 = (\epsilon + \delta, -\epsilon - \delta)$ for small $\delta > 0$.

Schedule σ_1 (sequential, A first): A acts, reaching $x'_A = (1-\alpha)(\epsilon+\delta)$. Then B acts, observing A's new position. Since B's policy drives it toward 0 but A has already moved toward 0, the separation $|x'_A - x'_B| = |(1-\alpha)(\epsilon+\delta) - (1-\alpha)(-\epsilon-\delta)| = 2(1-\alpha)(\epsilon+\delta) > 2\epsilon(1-\alpha) > \epsilon$ for $\alpha < 0.5$. Safety holds.

Schedule σ_2 (simultaneous): Both agents act concurrently. $x'_A = (1-\alpha)(\epsilon+\delta)$ and $x'_B = (1-\alpha)(-\epsilon-\delta)$. After sufficiently many simultaneous steps, both converge to 0, and $|x_A(t) - x_B(t)| = 2(1-\alpha)^t(\epsilon+\delta) \to 0$. For $t^* = \lceil \log(2\delta/\epsilon)/\log(1/(1-\alpha)) \rceil$, we have $|x_A(t^*) - x_B(t^*)| < \epsilon$, violating φ.

Since σ_1 and σ_2 are both consistent with the empty HB relation (no causal dependencies in this simple system), and $e_A \parallel e_B$, the triple (e_A, e_B, s_0) constitutes an interaction race. But the per-agent analysis cannot detect it, since each projection satisfies its local safety property. □

A.2 Proof of Theorem 2.5 (Statistical Intractability)

Proof. Let Σ denote the space of all HB-consistent schedules, equipped with the uniform measure μ. Let $R \subset \Sigma$ denote the set of schedules that trigger a race. By assumption, R is a smooth sub-manifold of codimension $k \geq 1$.

Measure of the race set. By the co-area formula, the μ-measure of the ϵ-neighborhood $R^\epsilon = \{s \in \Sigma : d(s, R) \leq \epsilon\}$ satisfies:

$$\mu(R^\epsilon) \leq C \cdot \epsilon^k \cdot \mathrm{vol}_{m-k}(R)$$

where $m = \dim(\Sigma)$ and C is a constant depending on the curvature of R.

Lipschitz constraint. Since policies are L-Lipschitz, two schedules within distance ϵ/L produce outcomes within distance ϵ in state space. Therefore, the set of schedules that produce *observably different* outcomes from the race threshold has measure at most $O((\epsilon/L)^k)$.

Sample complexity. To detect a race with probability at least $1-\delta$ using uniform sampling, we need at least N samples where:

$$(1-\mu(R^\epsilon))^N \le \delta \implies N \ge \frac{\ln(1/\delta)}{\mu(R^\epsilon)} \ge \frac{\ln(1/\delta)}{C\cdot(\epsilon/L)^k} = \Omega\left(\left(\frac{L}{\epsilon}\right)^k \ln\frac{1}{\delta}\right).$$

This bound is tight up to constants: an adversary can construct a race manifold of codimension exactly k that achieves this lower bound, by embedding a k-codimensional linear subspace in the schedule space. □

A.3 Proof of Theorem 2.6 (PSPACE-Hardness)

Proof. We reduce from the reachability problem for linearly-bounded one-counter automata, known to be PSPACE-complete [9].

Problem definition. A *bounded one-counter automaton* (BOCA) is a tuple (Q, q_0, q_f, B, Δ) where Q is a finite set of states, $q_0, q_f \in Q$ are initial and final states, $B \in \mathbb{N}$ is the counter bound, and $\Delta \subseteq Q \times \{-1, 0, +1\} \times Q$ is the transition relation. The reachability problem asks whether there exists a sequence of transitions from $(q_0, 0)$ to $(q_f, \cdot)$ such that the counter value remains in $[0, B]$ throughout.

Construction. Given a BOCA (Q, q_0, q_f, B, Δ), we construct a multi-agent system $\mathcal{M} = (\mathcal{A}, \mathcal{S}, \{\pi_i\}, T, \Sigma)$ as follows:

- Set $n = \lceil \log_2 B \rceil + 1$ agents and $d = |Q| + n$.
- The first $|Q|$ dimensions encode the automaton state as a one-hot vector; the remaining n dimensions encode the counter value in binary.
- For each transition $(q, \delta, q') \in \Delta$, construct a Lipschitz policy that, when agent i (corresponding to bit i of the counter) is scheduled, performs the appropriate bit flip with carry propagation.
- The safety predicate φ encodes that the automaton has not reached state q_f: $\varphi(s) = \mathbf{1}[s_{q_f} \le 0.5]$.

Correctness. A sequence of agent schedulings in $\mathcal{M}$ corresponds to a sequence of transitions in the BOCA: the relative ordering of bit-flip agents determines the counter increment/decrement behavior. An interaction race exists if and only if there is a scheduling (transition sequence) that reaches q_f—i.e., if and only if the BOCA reachability problem has a positive answer.

The policies are Lipschitz-continuous by construction: each policy computes a sigmoid-approximated step function with Lipschitz constant $O(|Q| \cdot B)$. Since the reduction runs in polynomial time and the BOCA reachability problem is PSPACE-complete, IRD for Lipschitz-continuous policies is PSPACE-hard. □

A.4 Proof of Theorem 2.7 (P-Membership for PWL)

Proof. Let each policy π_i be a ReLU network with L layers and at most W neurons per layer, so that π_i is piecewise-linear with at most $2^{W\cdot L}$ linear regions.

Step 1: Activation pattern enumeration. For a fixed input zonotope $Z_0 \subseteq \mathbb{R}^d$, propagating through a single ReLU layer with W neurons yields at most W "unstable" neurons (those with both positive and negative values in their range). The activation pattern $\alpha \in \{0, 1, \text{unstable}\}^W$ can be determined in $O(W \cdot p)$ time where p is the number of zonotope generators.

Step 2: HB constraint propagation. With HB constraints $\mathcal{C}$ on the generator coefficients, the feasible activation patterns are further restricted. For each unstable neuron k in agent i's policy, we solve a linear feasibility problem:

$$\exists \xi \in [-1, 1]^p : A\xi \leq b \text{ (HB constraints) and } c_k + g_k^T \xi \geq 0,$$

which determines whether the positive activation is feasible. This LP has $p + K$ constraints and can be solved in $O(p^2 K)$ time.

The key observation is that HB constraints typically eliminate most activation patterns: if events $e_i \rightarrow_{\text{hb}} e_j$, then the timing constraint $\xi_i \leq \xi_j - \delta_{\min}$ restricts the set of reachable states for agent j, potentially fixing the activation of several neurons.

Step 3: Per-region feasibility. For each feasible activation pattern, the joint dynamics become affine: $s_{t+1} = A_\alpha s_t + b_\alpha$ where A_α, b_α are determined by the activation pattern α. The race detection problem reduces to checking whether there exist two generator coefficient vectors $\xi_1, \xi_2 \in [-1, 1]^p$ satisfying $\mathcal{C}$ such that:

$$H(c + G\xi_1) \leq h \quad \text{and} \quad H(c + G\xi_2) \not\leq h.$$

This is a pair of LP feasibility problems, each solvable in polynomial time.

Complexity analysis. The total time is bounded by:

- Zonotope propagation through n policies of L layers each: $O(n \cdot L \cdot W \cdot p)$.
- HB constraint propagation at each layer: $O(n \cdot L \cdot W \cdot p^2 \cdot K)$.
- LP feasibility at the final layer: $O(p^3)$.

Since $p = O(n \cdot L \cdot W + p_0)$ and $K = O(n \cdot T)$ (number of HB edges), the overall complexity is $O(n^2 \cdot L^2 \cdot W^2 \cdot (n \cdot L \cdot W + T))$, which simplifies to $O(n^2 \cdot W^2 \cdot L)$ when $T = O(L)$ and treating n, L, W as the primary parameters. □

A.5 Proof of Theorem 3.5 (Analysis Convergence)

Proof. We prove convergence by showing that the abstract transfer function sequence is finite and monotone.

Generator count bound. At initialization, the zonotope has p_0 generators. Each ReLU layer introduces at most W_ℓ fresh generators (one per unstable neuron in layer ℓ). After processing all L layers of policy π_i:

$$p_i \leq p_0 + \sum_{\ell=1}^{L} W_\ell \leq p_0 + L \cdot W.$$

After processing all n agents:

$$p_{\text{total}} \leq p_0 + n \cdot L \cdot W.$$

With the zonotope reduction bound $p_{\max}$, we have $p \leq \max(p_0 + n \cdot L \cdot W, p_{\max})$.

Monotonicity. The abstract domain is ordered by set inclusion: $(Z_1, \mathcal{C}_1) \sqsubseteq (Z_2, \mathcal{C}_2)$ iff $\gamma(Z_1, \mathcal{C}_1) \subseteq \gamma(Z_2, \mathcal{C}_2)$. Each transfer function is monotone: if $Z_1 \sqsubseteq Z_2$, then $f^\sharp(Z_1) \sqsubseteq f^\sharp(Z_2)$. This holds for linear layers (exact), ReLU layers (the parallelotope relaxation is monotone), and HB constraint tightening (removing infeasible regions preserves the inclusion ordering).

Termination. Since each policy is a feedforward network (no loops), the analysis processes each layer exactly once in topological order. The total number of transfer function applications is $\sum_{i=1}^{n} L_i \leq n \cdot L$. No fixpoint iteration is needed for feedforward networks.

Soundness. By induction on the layer index. Base case: the initial zonotope $Z_0 = \mathsf{Zono}(s_0, 0)$ exactly represents the initial state. Inductive step: if Z_ℓ soundly over-approximates the reachable states at layer ℓ, then $f_\ell^\sharp(Z_\ell)$ soundly over-approximates the reachable states at layer $\ell+1$, since each transfer function is a sound abstraction of the corresponding concrete operation (linear map, ReLU). The HB constraints only tighten the over-approximation (removing infeasible states), preserving soundness by Theorem 3.4. □

A.6 Proof of Theorem 5.2 (Pipeline Soundness)

Proof. We must show that if the pipeline reports no race, then no HB-consistent schedule from the analyzed initial region produces a safety violation.

The proof proceeds by composing the soundness of each pipeline stage:

Stage 1 (Traces). Trace collection is used only for HB graph construction and calibration; it does not affect soundness.

Stage 2 (HB Graph). The HB graph is constructed conservatively: an edge $e_i \rightarrow_{\text{hb}} e_j$ is added only when physical causality is detected with high confidence. Omitting an edge (treating causally ordered events as concurrent) *enlarges* the set of considered schedules, preserving soundness (though potentially increasing false positives).

Stage 3 (Decomposition). By Theorem F.2 (Appendix F), assume-guarantee decomposition is sound: if each interaction group satisfies its local safety property under its assumed environment behavior, then the full system satisfies the global safety property.

Stage 4 (Abstract Interpretation). By Theorem 3.5, the constrained zonotope analysis produces a sound over-approximation of the reachable states for each interaction group.

Stage 5 (CEGAR). By Theorem 4.2, the CEGAR loop preserves soundness: it never classifies a true race as spurious. Only spurious candidates are eliminated.

Stage 6 (MCTS). MCTS explores scheduling decisions within the refined over-approximation. HB-consistency pruning only removes schedules that violate the HB partial order—these are physically infeasible and could not produce real races.

Stage 7 (IS). Importance sampling provides probability estimates for detected races; it does not affect the soundness of race detection.

Composing these guarantees: if no race is found in the refined over-approximation (which contains all concrete reachable states), then no race exists. □

B CEGAR Soundness Proof (Extended)

We provide a detailed proof of the CEGAR soundness and termination theorems stated in Section 4.

B.1 Detailed Proof of Theorem 4.2

Proof. We prove the three claims of the theorem separately.

Claim 1: Spurious classification is sound. Suppose the CEGAR loop classifies candidate $(e, e', \hat{s})$ as Spurious. This occurs when ExtractWitness returns $\perp$, meaning the refined constraint set $\mathcal{C}'$ admits no $(\xi_1^*, \xi_2^*) \in [-1, 1]^{2p}$ satisfying:

$$\mathcal{C}'(\xi_1) \wedge \mathcal{C}'(\xi_2) \wedge H(c + G\xi_1) \leq h \wedge \exists j : H_j(c + G\xi_2) > h_j.$$

We must show that $\mathcal{C}' \supseteq \mathcal{C}$ does not exclude any concrete reachable state. Each constraint added during refinement has the form $v^T G\xi \leq \beta - v^T c$, derived from a separating predicate $P(s) = (v^T s \leq \beta)$ that was verified to hold for all concrete reachable states in the relevant scheduling window. Formally, for the schedule σ corresponding to timing ξ:

$$v^T \text{reach}(\sigma, c + G\xi) \leq \beta \quad \forall \xi \in \gamma_\xi(\mathcal{C}) \text{ such that } (c + G\xi, \sigma) \text{ is concrete.}$$

Since every concrete state satisfies P, removing abstract states that violate P cannot remove any concrete state. By induction on the number of refinement iterations, $\mathcal{C}'$ admits all concrete reachable states. Therefore, if $\mathcal{C}'$ admits no race witness, no concrete race witness exists.

Claim 2: True race classification is sound. A candidate is classified as True Race only when $\varphi(\text{reach}(\sigma_1, s_1)) = 1$ and $\varphi(\text{reach}(\sigma_2, s_2)) = 0$ for concrete states s_1, s_2 and HB-consistent schedules σ_1, σ_2. This directly satisfies Theorem 2.3: the events e, e' are concurrent (inherited from the abstract candidate), the schedules differ in the ordering of e, e', and the safety outcomes differ.

Claim 3: No false negatives. Every true race produces a candidate in the abstract interpretation (by soundness of the zonotope, Theorem 3.5). The CEGAR loop processes each candidate and either confirms it (returning True Race) or attempts to refute it. For a true race, the concrete simulation at Step 2 of Algorithm 1 will find $\varphi(s_1') = 1$ and $\varphi(s_2') = 0$ (since the abstract witness, being concretizable, produces a concrete race), and the loop returns True Race in the first iteration. Thus, no true race is ever classified as spurious. □

B.2 Detailed Proof of Theorem 4.3

Proof. We show that the CEGAR loop makes strictly monotone progress in each iteration, within a finite domain.

Progress measure. Define $V_k = \mathrm{vol}(\gamma(Z, \mathcal{C}_k))$ as the volume of the concretization of the constrained zonotope at iteration k. Each refinement adds a constraint that eliminates a region of positive volume (the spurious race region contains a neighborhood of the abstract witness ξ^*, which has positive volume in the generator space). Therefore, $V_{k+1} < V_k$ strictly.

Finiteness for PWL policies. For piecewise-linear policies, the abstract domain can be decomposed into at most $R = \prod_{i=1}^{n} \prod_{\ell=1}^{L} 2^{W_\ell}$ regions, each corresponding to a unique activation pattern. Within each region, the dynamics are affine, and the race detection problem is a linear feasibility problem with a unique answer (feasible or infeasible). Neuron-split refinement (Strategy 2) resolves one unstable neuron per iteration, reducing the number of undecided regions. After at most $\sum_{i=1}^{n} \sum_{\ell=1}^{L} W_\ell \leq nLW$ neuron splits, all neurons are resolved and the abstract domain perfectly tracks the concrete dynamics. At this point, every remaining candidate either has a concrete witness or is provably infeasible.

Practical convergence. In experiments, convergence occurs in at most 3 iterations per candidate (average 1.8), far fewer than the worst-case nLW bound. This is because predicate-guided constraint injection (Strategy 1) often suffices to eliminate spurious regions without explicit neuron splitting. □

C Specification Language

MARACE accepts safety specifications in a domain-specific language (DSL) designed for multi-agent temporal properties. We define the syntax and semantics formally.

C.1 BNF Grammar

The specification language is defined by the following BNF grammar:

```
<spec>       ::= <safety_spec> | <liveness_spec> | <spec> "and" <spec>
<safety_spec> ::= "always" <predicate>
               |  "always" <predicate> "within" <time_bound>
<liveness_spec> ::= "eventually" <predicate>
               |  <predicate> "responds" <predicate> "within" <time_bound>
<predicate>  ::= <linear_pred> | <distance_pred> | <region_pred>
               |  <predicate> "and" <predicate>
               |  <predicate> "or" <predicate>
               |  "not" <predicate>
<linear_pred>   ::= <expr> <cmp_op> <expr>
<distance_pred> ::= "dist(" <agent_ref> "," <agent_ref> ")" <cmp_op> <num>
<region_pred>   ::= <agent_ref> "in" <region>
<agent_ref>     ::= "agent[" <int> "]" "." <field>
<field>         ::= "x" | "y" | "vx" | "vy" | "theta" | <string>
<region>        ::= "box(" <num> "," <num> "," <num> "," <num> ")"
               |  "circle(" <num> "," <num> "," <num> ")"
<expr>       ::= <num> | <agent_ref> | <expr> <arith_op> <expr>
<cmp_op>     ::= "<" | "<=" | ">" | ">=" | "=="
<arith_op>   ::= "+" | "-" | "*"
<time_bound> ::= <int> "steps"
<num>        ::= <float> | <int>
```

C.2 Denotational Semantics

We define the semantics of specifications over infinite traces $\tau = s_0, s_1, s_2, \ldots$ where each $s_t \in \mathcal{S}$.

Definition C.1 (Trace satisfaction)**.** *The satisfaction relation $\tau, t \models \psi$ is defined inductively:*

$$\tau, t \models \text{“}\textit{always } \psi\text{”} \iff \forall t' \geq t.\ \tau, t' \models \psi \tag{3}$$
$$\tau, t \models \text{“}\textit{eventually } \psi\text{”} \iff \exists t' \geq t.\ \tau, t' \models \psi \tag{4}$$
$$\tau, t \models \text{“}\psi_1 \textit{ responds } \psi_2 \textit{ within } k\text{”} \iff \tau, t \models \psi_1 \implies \exists t' \in [t, t+k].\ \tau, t' \models \psi_2 \tag{5}$$
$$\tau, t \models \psi_1 \textit{ and } \psi_2 \iff \tau, t \models \psi_1 \textit{ and } \tau, t \models \psi_2 \tag{6}$$
$$\tau, t \models \psi_1 \textit{ or } \psi_2 \iff \tau, t \models \psi_1 \textit{ or } \tau, t \models \psi_2 \tag{7}$$
$$\tau, t \models \textit{not } \psi \iff \tau, t \not\models \psi \tag{8}$$

For atomic predicates:

$$\tau, t \models e_1 \leq e_2 \iff e_{1 s_t} \leq e_{2 s_t} \tag{9}$$
$$\tau, t \models \textit{dist}(i, j) > r \iff \|s_t^{(i)} - s_t^{(j)}\| > r \tag{10}$$
$$\tau, t \models i \textit{ in } R \iff s_t^{(i)} \in R \tag{11}$$

where e_s denotes the evaluation of expression e in state s, and $s_t^{(i)}$ is agent i's state component at time t.

C.3 Robustness Semantics

For quantitative analysis, we extend the Boolean semantics to a *robustness* semantics $\rho(\tau, t, \psi) \in \mathbb{R}$ following the signal temporal logic (STL) tradition:

$$\rho(\tau, t, e_1 \leq e_2) = e_{2 s_t} - e_{1 s_t} \tag{12}$$
$$\rho(\tau, t, \text{always } \psi) = \inf_{t' \geq t} \rho(\tau, t', \psi) \tag{13}$$
$$\rho(\tau, t, \text{eventually } \psi) = \sup_{t' \geq t} \rho(\tau, t', \psi) \tag{14}$$
$$\rho(\tau, t, \psi_1 \text{ and } \psi_2) = \min(\rho(\tau, t, \psi_1), \rho(\tau, t, \psi_2)) \tag{15}$$
$$\rho(\tau, t, \text{not } \psi) = -\rho(\tau, t, \psi) \tag{16}$$

The robustness value is positive when the specification is satisfied and negative when violated. The safety margin reported by Marace (Theorem 5.1) corresponds to the robustness at the worst-case schedule.

C.4 Compilation to Linear Constraints

Safety specifications involving linear predicates compile directly to the constraint form $Hs \leq h$ used by the abstract interpreter. Distance predicates are linearized via the inscribed polytope approximation: $\|x\| > r$ is approximated by $\bigvee_{k=1}^{K} v_k^T x > r$ where $\{v_k\}$ are uniformly spaced unit vectors.

D Probabilistic Estimation

D.1 Deployment Schedule Distributions

In deployed MARL systems, the execution schedule is not adversarial but follows a *deployment distribution* μ_D determined by the system's scheduling infrastructure (e.g., round-robin, event-driven, or real-time scheduling). We model μ_D as a distribution over HB-consistent schedules.

Definition D.1 (Race probability)**.** *Given a deployment distribution μ_D over schedules, the* race probability *for an interaction race (e, e', s_0) is:*

$$p_R = \mathbb{P}_{\sigma\sim\mu_D}[\varphi(\text{reach}(\sigma, s_0)) = 0 \text{ and } e \text{ races with } e' \text{ under } \sigma].$$

D.2 Importance Sampling Estimator

Direct Monte Carlo estimation of p_R is inefficient when p_R is small (Theorem 2.5). We use importance sampling with a proposal distribution q that concentrates mass near the race manifold:

Definition D.2 (IS estimator)**.** *The self-normalized importance sampling estimator is:*

$$\hat{p}_R^{IS} = \frac{\sum_{j=1}^N w_j \cdot \mathbf{1}[\text{race under } \sigma_j]}{\sum_{j=1}^N w_j}, \quad w_j = \frac{\mu_D(\sigma_j)}{q(\sigma_j)}, \quad \sigma_j \sim q.$$

Theorem D.3 (IS consistency)**.** *If $q(\sigma) > 0$ whenever $\mu_D(\sigma) > 0$ and σ is in the race set, then $\hat{p}_R^{IS} \xrightarrow{a.s.} p_R$ as $N \to \infty$.*

Proof. By the strong law of large numbers applied to the numerator and denominator separately. The numerator converges to $\mathbb{E}_q[w \cdot \mathbf{1}[\text{race}]] = \mathbb{E}_{\mu_D}[\mathbf{1}[\text{race}]] = p_R$, and the denominator converges to $\mathbb{E}_q[w] = 1$. Since both converge almost surely and the denominator is bounded away from zero (as q has full support on the μ_D-support), the ratio converges to p_R. □

D.3 Concentration Bounds

We provide finite-sample confidence intervals using the self-normalized bound:

Theorem D.4 (Self-normalized concentration)**.** *Let $w_1, \ldots, w_N$ be importance weights satisfying $\max_j w_j/\bar{w} \le M$ where $\bar{w} = N^{-1}\sum_j w_j$. Then for any $\delta > 0$:*

$$\mathbb{P}\big[|\hat{p}_R^{IS} - p_R| > \epsilon\big] \le 2\exp\left(-\frac{N_{\text{eff}}\epsilon^2}{2M^2}\right)$$

where $N_{\text{eff}} = (\sum_j w_j)^2/\sum_j w_j^2$ is the effective sample size.

Proof. The self-normalized estimator satisfies a Bernstein-type inequality. Following Owen [34], the variance of $\hat{p}_R^{\text{IS}}$ is bounded by:

$$\text{Var}[\hat{p}_R^{\text{IS}}] \le \frac{p_R(1-p_R)}{N_{\text{eff}}}.$$

The weight truncation $\max w_j/\bar{w} \le M$ ensures sub-exponential tails, and Bernstein's inequality yields the exponential concentration. Specifically, define $Y_j = w_j(\mathbf{1}[\text{race under } \sigma_j] - p_R)/\bar{w}$. Then $\mathbb{E}[Y_j] = 0$, $|Y_j| \le M$, and:

$$\mathbb{P}\left[|[|\right]\sum_{j=1}^N Y_j > N\epsilon\right] \le 2\exp\left(-\frac{N^2\epsilon^2}{2\sum_j \mathbb{E}[Y_j^2] + 2NM\epsilon/3}\right).$$

Since $\sum_j \mathbb{E}[Y_j^2] \leq NM^2/N_{\text{eff}} \cdot N$, simplification gives the claimed bound. □

In our experiments, the effective sample size N_{eff} equals 50 (the full sample count), indicating that the proposal distribution q closely matches the target μ_D restricted to the race set, yielding tight confidence intervals $[1.0, 1.0]$ for the planted races.

D.4 Cross-Entropy Convergence

The proposal distribution q is optimized via the cross-entropy (CE) method [33]. Starting from a uniform proposal q_0 over HB-consistent schedules, the CE method iteratively:

1. Draws N samples from q_t;
2. Selects the top-ρ fraction ("elite samples") by race severity;
3. Fits q_{t+1} to the elite samples by minimizing KL divergence.

Proposition D.5 (CE convergence)**.** *Under regularity conditions (the parametric family of proposals includes the optimal proposal q^*, and the elite threshold is chosen adaptively), the CE iterates satisfy $D_{KL}(q^* \| q_t) \to 0$ at rate $O(1/\sqrt{t})$.*

The proof follows the standard CE convergence theory [33] adapted to the constraint that proposals must assign zero mass to HB-inconsistent schedules.

E Lipschitz Analysis

The Lipschitz constant of agent policies plays a central role in both the statistical intractability result (Theorem 2.5) and the precision of the abstract domain.

E.1 Spectral Norm Bounds

For a ReLU network $\pi = W_L \circ \sigma \circ W_{L-1} \circ \cdots \circ \sigma \circ W_1$ where σ is the ReLU activation, the global Lipschitz constant satisfies:

$$\text{Lip}(\pi) \leq \prod_{\ell=1}^{L} \|W_\ell\|_2.$$

This spectral norm product bound is easy to compute but often loose. We compute $\|W_\ell\|_2$ via power iteration (10 iterations suffice for convergence to relative error $< 10^{-6}$ in all our experiments).

E.2 LipSDP Bounds

For tighter bounds, we use the LipSDP framework [30], which formulates the Lipschitz constant computation as a semidefinite program:

$$\min_{\gamma, T} \quad \gamma^2 \tag{17}$$

$$\text{s.t.} \quad \begin{bmatrix} W_1^T T W_1 - \gamma^2 I & W_1^T T \\ T W_1 & -T + W_2^T W_2 \end{bmatrix} \preceq 0, \tag{18}$$

$$T = \text{diag}(t_1, \ldots, t_W), \quad t_i \geq 0. \tag{19}$$

This yields the tightest known polynomial-time computable bound on $\text{Lip}(\pi)$ for ReLU networks.

E.3 Lipschitz Calibration

In MARACE, the Lipschitz constant is used to calibrate the abstraction error. Given $\text{Lip}(\pi_i)$ for each agent, the maximum state perturbation from a schedule change of magnitude Δt is bounded by:

$$\|\Delta s_i\| \leq \text{Lip}(\pi_i) \cdot \text{Lip}(T) \cdot \Delta t$$

where $\text{Lip}(T)$ is the Lipschitz constant of the transition function.

Proposition E.1 (Calibration convergence)**.** *As the number of calibration traces $K \to \infty$, the empirical Lipschitz estimate $\hat{L}_K$ satisfies:*

$$\hat{L}_K \leq \text{Lip}(\pi) \leq \hat{L}_K + O\left(\sqrt{\frac{\log K}{K}}\right)$$

with probability at least $1 - 1/K$.

Proof. The empirical Lipschitz estimate is:

$$\hat{L}_K = \max_{1 \leq j < k \leq K} \frac{\|\pi(s_j) - \pi(s_k)\|}{\|s_j - s_k\|}.$$

This is a lower bound on $\text{Lip}(\pi)$ by definition. The upper bound follows from a covering number argument: with K samples from a compact domain, the uncovered volume has measure $O(K^{-1/d})$, and the maximum Lipschitz ratio over the uncovered region is bounded by the global Lipschitz constant plus a term that decreases as $O(\sqrt{\log K/K})$ by a uniform convergence argument over Lipschitz functions. □

E.4 Abstract Error Propagation

The abstraction error ϵ_ℓ at layer ℓ of the zonotope analysis satisfies the recurrence:

$$\epsilon_{\ell+1} \leq \text{Lip}(f_\ell) \cdot \epsilon_\ell + \delta_\ell$$

where δ_ℓ is the local approximation error introduced by the ReLU relaxation at layer ℓ. For the parallelotope relaxation, $\delta_\ell \leq \sum_{k: l_k < 0 < u_k} |u_k l_k|/(2(u_k - l_k))$.

The total abstraction error after L layers is bounded by:

$$\epsilon_L \leq \epsilon_0 \prod_{\ell=0}^{L-1} \text{Lip}(f_\ell) + \sum_{\ell=0}^{L-1} \delta_\ell \prod_{j=\ell+1}^{L-1} \text{Lip}(f_j).$$

F Compositional Decomposition

F.1 Assume-Guarantee Framework

Compositional verification decomposes the n-agent system into G interaction groups $\mathcal{G}_1, \ldots, \mathcal{G}_G$ where $\bigcup_g \mathcal{G}_g = \mathcal{A}$ and agents in different groups have limited interaction.

Definition F.1 (Interaction group contract)**.** *An* interaction group contract *for group $\mathcal{G}_g$ is a pair* $(\mathsf{Assume}_g, \mathsf{Guarantee}_g)$ *where:*

- $\mathsf{Assume}_g \subseteq \mathcal{S}$ *specifies the assumed behavior of agents outside $\mathcal{G}_g$ (their states remain within* Assume_g*);*

- $\mathsf{Guarantee}_g \subseteq \mathcal{S}$ *specifies the guaranteed behavior of agents within* $\mathcal{G}_g$ *(their states remain within* $\mathsf{Guarantee}_g$*).*

Theorem F.2 (Assume-guarantee soundness)**.** *If for each group* g*:*
(i) Under assumption Assume_g*, the group* $\mathcal{G}_g$ *satisfies* $\mathsf{Guarantee}_g$*; and*
(ii) $\mathsf{Guarantee}_h \subseteq \mathsf{Assume}_g$ *for all groups* $h \neq g$*;*
then the full system satisfies $\bigcap_g \mathsf{Guarantee}_g$*.*

Proof. By induction on the number of execution steps. At step $t = 0$, all agents are in their initial states, which satisfy all assumptions by hypothesis.

Assume at step t, all agents satisfy their group guarantees. For each group g, the agents outside $\mathcal{G}_g$ are in $\bigcup_{h \neq g} \mathsf{Guarantee}_h \subseteq \mathsf{Assume}_g$ by condition (ii). Therefore, by condition (i), the agents in $\mathcal{G}_g$ at step $t + 1$ satisfy $\mathsf{Guarantee}_g$.

By induction, all agents satisfy their guarantees at all steps. □

F.2 Group Size Theory

The effectiveness of decomposition depends on the maximum interaction group size, which determines the dimensionality of the per-group verification problem.

Definition F.3 (Interaction graph)**.** *The* interaction graph $I = (\mathcal{A}, E_I)$ *has an edge* $(i, j) \in E_I$ *if agents* i *and* j *have concurrent events that could affect a shared safety predicate. The interaction groups are the connected components of* I*.*

Theorem F.4 (Group size bound)**.** *If the interaction graph has maximum degree* Δ *and the safety predicates have locality radius* r *(each predicate involves agents within distance* r *in the interaction graph), then the maximum interaction group size is bounded by* $O(\Delta^r)$*.*

Proof. The interaction group containing agent i is the connected component reachable from i in the interaction graph restricted to agents participating in shared safety predicates. From any vertex in a graph of maximum degree Δ, the number of vertices reachable within distance r is at most $1 + \Delta + \Delta^2 + \cdots + \Delta^r = O(\Delta^r)$. □

In spatially distributed multi-agent systems, Δ and r are typically small constants (each agent interacts with a bounded number of neighbors), yielding constant-size groups and enabling the linear scaling observed in our experiments.

F.3 Percolation Bounds

For randomly distributed agents, we can obtain probabilistic bounds on group sizes using percolation theory.

Proposition F.5 (Sub-critical percolation)**.** *If* n *agents are distributed uniformly at random in* $[0, 1]^2$ *with interaction radius* $r_n = c\sqrt{\log n / n}$ *for* $c < 1/\sqrt{\pi}$*, then with probability* $1 - O(1/n)$*, the maximum interaction group size is* $O(\log n)$*.*

Proof. This follows from the theory of random geometric graphs. The expected degree is $\lambda = n\pi r_n^2 = \pi c^2 \log n$. For $\pi c^2 < 1$, the random geometric graph is in the sub-critical regime, where the largest connected component has size $O(\log n)$ with high probability. The result follows from the standard percolation threshold for two-dimensional Poisson Boolean models, adapted to the finite setting via a union bound over n agents. □

G Proof Certificates

MARACE can produce machine-checkable proof certificates that independently verify the absence of interaction races.

G.1 Certificate Format

A proof certificate $\mathcal{P}$ for race absence consists of:

Definition G.1 (Proof certificate)**.** $\mathcal{P} = (\mathcal{Z}_0, \{f_\ell^\sharp\}_{\ell=1}^{L_{total}}, \mathcal{C}_{HB}, \mathcal{C}_{CEGAR}, m^*)$ *where:*

- $\mathcal{Z}_0 = \mathsf{Zono}(c_0, G_0)$ *is the initial zonotope covering the analyzed state region;*
- $\{f_\ell^\sharp\}$ *is the sequence of abstract transfer functions (linear maps and ReLU relaxation coefficients);*
- $\mathcal{C}_{HB}$ *is the set of HB constraints on generator coefficients;*
- $\mathcal{C}_{CEGAR}$ *is the set of refinement constraints added by the CEGAR loop;*
- $m^* > 0$ *is the certified minimum safety margin.*

G.2 Verification Algorithm

The certificate is verified by an independent checker that replays the abstract computation and confirms the safety margin:

Algorithm 2 Certificate Verification

Require: Certificate $\mathcal{P} = (\mathcal{Z}_0, \{f_\ell^\sharp\}, \mathcal{C}_{\text{HB}}, \mathcal{C}_{\text{CEGAR}}, m^*)$, safety predicate (H, h)
Ensure: Accept or Reject

```
Z ← Z_0; C ← C_HB ∪ C_CEGAR
for ℓ = 1, ..., L_total do
    Z ← f_ℓ^♯(Z)                                   ▷ Apply transfer function
    Verify f_ℓ^♯ is a sound abstraction of f_ℓ
end for
for each constraint c ∈ C_CEGAR do
    Verify c was derived from a valid separating predicate
end for
Compute m_verified ← min_k min_{ξ∈[−1,1]^p, C}(h_k − H_k(c + Gξ))/‖H_k‖
if m_verified ≥ m* and m* > 0 then
    return Accept
else
    return Reject
end if
```

Proposition G.2 (Certificate soundness)**.** *If Algorithm 2 accepts certificate $\mathcal{P}$, then the multi-agent system has no interaction race for the specified safety predicate and initial state region.*

Proof. The verifier independently confirms that: (1) each transfer function is a sound over-approximation of the corresponding concrete operation; (2) the HB constraints are consistent with the declared happens-before graph; (3) each CEGAR refinement constraint was derived from a valid separating predicate (one that holds for all concrete reachable states); (4) the minimum safety margin over all feasible states in the final zonotope is positive. If all four hold, no concrete reachable state violates the safety predicate. □

G.3 Certificate Size

The certificate size is polynomial in the system parameters: $O(n \cdot L \cdot W^2 + n^2 \cdot T + R \cdot d)$ where the first term accounts for the weight matrices and relaxation coefficients, the second for the HB constraints, and the third for the CEGAR refinement constraints (R refinements, each adding a d-dimensional constraint). For our experimental configurations, certificates range from approximately 2 KB (2-agent highway) to 18 KB (8-agent warehouse with CEGAR constraints).

H Additional Experimental Details

H.1 Policy Architectures

Highway intersection. Each vehicle agent uses a 2-layer ReLU network: input dimension 4 (position x, y, velocity v_x, v_y), hidden layer of 64 units, output dimension 2 (acceleration, steering). Total parameters: 450.

Warehouse corridor. Each robot agent uses a 2-layer ReLU network: input dimension 6 (position x, y, velocity v_x, v_y, orientation θ, load status), hidden layer of 128 units, output dimension 3 (forward velocity, angular velocity, gripper). Total parameters: 1,155.

H.2 Training Details

Policies are trained using Proximal Policy Optimization (PPO) [27] with independent learning (each agent optimizes its own reward). Training hyperparameters:

- Learning rate: 3×10^{-4} with linear decay;
- Discount factor: $\gamma = 0.99$;
- GAE parameter: $\lambda = 0.95$;
- Clip ratio: $\epsilon = 0.2$;
- Training episodes: 10,000 per agent;
- Batch size: 64 episodes.

H.3 Race Planting Procedure

To create ground-truth interaction races for evaluation, we use the following procedure:

1. **Identify critical regions:** Determine spatial regions where multiple agents' paths intersect (intersection center, corridor junctions).
2. **Construct race-inducing initial conditions:** Place agents at positions and velocities such that simultaneous action produces collision but sequential (HB-ordered) action allows safe passage.
3. **Verify ground truth:** Exhaustively enumerate all HB-consistent schedules for the initial condition (feasible for $n \leq 4$) and confirm that exactly one pair of events constitutes a race.
4. **Embed in trace set:** Include the race-inducing initial condition among the $K = 5$ trace episodes, with the remaining 4 episodes drawn from the normal training distribution.

For the warehouse corridor benchmark with $n \geq 6$ agents, exhaustive enumeration is infeasible, so ground truth is established by combining manual analysis with targeted simulation (1000 random schedules for each candidate race pair).

H.4 CEGAR Implementation Details

The CEGAR loop is implemented as follows:

Witness extraction. ExtractWitness solves a linear program over the generator coefficients to find (ξ_1^*, ξ_2^*) that maximize the safety margin difference $m(\sigma_1) - m(\sigma_2)$ subject to the constraints $\mathcal{C}'$ and the condition that σ_1 orders e before e' while σ_2 orders e' before e. This LP has $2p + 2K'$ variables and constraints, where K' is the refined constraint count.

Spuriousness diagnosis. DiagnoseSpurious computes the *abstraction gap*: the difference between the abstract safety margin (computed from the zonotope) and the concrete safety margin (computed from simulation). The separating predicate P is derived from the normal to the safety boundary at the concrete reachable state, projected onto the generator space.

Refinement constraint generation. RefineConstraints generates constraints by: (a) computing the gradient $\nabla_\xi m(\sigma)$ of the safety margin with respect to the generator coefficients at the spurious witness; (b) constructing the separating hyperplane $\nabla_\xi m \cdot \xi \leq \nabla_\xi m \cdot \xi^* + \epsilon_{\text{sep}}$ where ϵ_{sep} is a small margin ensuring strict separation.

H.5 Time Breakdown

Table 5 provides a detailed breakdown of analysis time for each pipeline stage, including the CEGAR stage.

Table 5: Time breakdown by pipeline stage (seconds) with CEGAR enabled. Percentages show fraction of total time.

		Pipeline Stage							
Scenario	n	Trace	HB	Decomp	Abstract	CEGAR	MCTS	IS	**Total**
Highway	2	0.005	0.001	0.001	0.001	0.000	0.009	0.001	0.018
Highway	3	0.013	0.001	0.002	0.001	0.000	0.022	0.001	0.040
Highway	4	0.025	0.001	0.003	0.001	0.000	0.044	0.002	0.076
Warehouse	4	0.020	0.004	0.002	0.001	0.012	0.027	0.002	0.068
Warehouse	6	0.058	0.006	0.004	0.001	0.042	0.065	0.004	0.18
Warehouse	8	0.120	0.008	0.006	0.002	0.098	0.172	0.004	0.41

In the highway benchmark, the CEGAR stage has zero overhead because no spurious candidates are generated. In the warehouse benchmark, CEGAR accounts for 18–24% of total time—a modest cost for eliminating all false positives. The dominant costs remain trace collection and MCTS search.

H.6 Zonotope Statistics

The HB graphs are dense (95% of event pairs are causally ordered), reflecting the tight physical coupling in both benchmarks. The coverage percentage (fraction of the state space covered by the zonotope) ranges from 2.09% to 5.45%, indicating that the abstract domain effectively focuses on the relevant portion of the state space. The abstraction error grows sub-linearly with agent count, consistent with the theoretical bound from Section E.

Table 6: Zonotope propagation statistics for the detection experiments.

Scenario	n	**HB Events**	**HB Edges**	**Coverage (%)**	**Abs. Error**
Highway	2	300	285	3.62	0.036
Highway	3	400	380	3.98	0.040
Highway	4	500	475	4.71	0.047
Warehouse	4	500	475	2.09	0.047
Warehouse	6	700	665	3.21	0.071
Warehouse	8	900	855	5.45	0.121

I Zonotope Reduction

As the analysis propagates through ReLU layers, the number of zonotope generators grows. To maintain computational tractability, we periodically reduce the generator count.

I.1 Reduction Strategy

Given a zonotope $Z = \mathsf{Zono}(c, G)$ with $p > p_{\max}$ generators, we reduce to $p_{\max}$ generators using the following procedure:

1. **Sort generators by norm:** order $g_1, \ldots, g_p$ such that $\|g_1\| \geq \|g_2\| \geq \cdots \geq \|g_p\|$.
2. **Keep top generators:** retain $g_1, \ldots, g_{p_{\max}-d}$.
3. **Merge remaining generators:** replace $g_{p_{\max}-d+1}, \ldots, g_p$ with d axis-aligned generators forming the interval hull:

$$g'_k = \left(\sum_{j=p_{\max}-d+1}^{p} |g_{jk}| \right) e_k, \quad k = 1, \ldots, d.$$

I.2 Bounded Error Analysis

Theorem I.1 (Reduction error bound)**.** *Let $Z = \mathsf{Zono}(c, G)$ be a zonotope and $Z' = \mathsf{Zono}(c, G')$ the reduced zonotope obtained by the above procedure. Then:*

$$Z \subseteq Z' \subseteq Z \oplus B_\infty(\epsilon_{red})$$

where $\epsilon_{red} = \max_k \sum_{j=p_{\max}-d+1}^{p} |g_{jk}|$ and $B_\infty(\epsilon)$ is the ℓ_∞-ball of radius ϵ.

Proof. Containment $Z \subseteq Z'$. For any $s = c + \sum_j \xi_j g_j \in Z$ with $\xi \in [-1, 1]^p$, we construct $\xi' \in [-1, 1]^{p_{\max}}$ such that $s = c + G'\xi'$. For the retained generators, $\xi'_j = \xi_j$ for $j \leq p_{\max} - d$. For the merged generators, $\xi'_{p_{\max}-d+k} = \sum_{j > p_{\max}-d} \xi_j g_{jk} / g'_k$ for each axis k. By the triangle inequality:

$$|\xi'_{p_{\max}-d+k}| = \frac{|[|]\sum_j \xi_j g_{jk}}{\sum_j |g_{jk}|} \leq \frac{\sum_j |g_{jk}|}{\sum_j |g_{jk}|} = 1.$$

Hence $s \in Z'$.

Over-approximation bound. For any $s' = c + G'\xi' \in Z'$, we can write $s' = c + \sum_{j=1}^{p_{\max}-d} \xi'_j g_j + \sum_{k=1}^{d} \xi'_{p_{\max}-d+k} g'_k e_k$. The last d terms contribute at most $|g'_k|$ in each dimension k, while the corresponding terms in Z contribute at most $\sum_{j > p_{\max}-d} |g_{jk}| = g'_k$. The over-approximation error is thus exactly $\epsilon_{\text{red}} = \max_k g'_k$ per dimension. □

I.3 Adaptive Reduction

In practice, we use an adaptive reduction strategy:

- **Threshold:** Reduction is triggered when $p > 2p_{\max}$ to amortize the cost.
- **PCA-based merging:** Instead of axis-aligned merging, we compute the principal components of the small generators and merge along the top-d principal directions. This reduces the approximation error when the small generators are correlated.
- **Error tracking:** The cumulative reduction error is tracked and reported as part of the abstract error (Table 6).

Proposition I.2 (PCA merging improvement)**.** *PCA-based merging reduces the over-approximation volume by a factor of at most $\prod_{k=1}^{d} \sigma_k/\sigma'_k$ compared to axis-aligned merging, where σ_k and σ'_k are the singular values of the small generator matrix in the original and rotated coordinate systems, respectively.*

Proof. The volume of the over-approximation box in the axis-aligned case is $\prod_k 2\sum_j |g_{jk}|$. In the PCA-rotated coordinate system, the box volume is $\prod_k 2\sigma_k$ where σ_k are the singular values of $[g_{p_{\max}-d+1} \cdots g_p]$. Since the singular values satisfy $\sigma_k \leq \sum_j |g_{jk}|$ with equality only when all small generators are axis-aligned, PCA merging always yields equal or smaller over-approximation volume. □

For the configurations in our experiments, PCA merging was not necessary (the generator count stayed below $p_{\max} = 100$ throughout), but the mechanism is available for larger networks with more unstable neurons.